

4. Container 빌드 & 저장하기

관련 내용

1. Docker, Podman, Skopeo 비교

- Docker
 - Dockerd라는 데몬이 항상 백그라운드에서 실행되며, CLI는 이 데몬에게 명령을 전달하는 클라이언트 역할
 - ※ Daemon: 백그라운드에서 사용자와 상호작용 없이 요청이 들어오면 처리하는 프로그램
 - 이미지 빌드, 컨테이너 실행, 네트워크/볼륨 관리 등 컨테이너 라이프사이클 전체를 담당

```
docker CLI (조종사가 보내는 요청)
↓
dockerd (관제탑 = 데몬, 항상 떠있음)
↓
containerd → runc (실제 항공기 이착륙 = 컨테이너 실행)
```

- Podman
 - 데몬이 없음. podman run을 실행하면 프로세스 자체가 컨테이너의 부모 프로세스가 됨.
 - 즉, Docker처럼 중앙 데몬에 요청을 보내는 구조가 아님.
 - rootless 컨테이너를 지원하여 보안측면에서 강점이 있음.
- Skopeo
 - 이미지 “실행”하지 않고 레지스트리간 이미지를 복사, 검사, 서명하는데 특화된 도구
 - 대표 용도

- 레지스트리 간 이미지 복사: 레지스트리 A → 로컬에 pull 받지 않고 레지스트리 B로 전송
- 이미지 메타데이터 검사 : 이미지를 실제로 받지 않고 태그, 레이어 정보 등 확인
- 이미지 삭제, 서명 검증 등

구분	Docker	Podman	Skopeo
주요 역할	이미지 빌드 + 실행 + 관리 (풀스택)	이미지 빌드 + 실행 + 관리 (Docker 대체)	이미지 전송/검사 전용 (실행 안 함)
아키텍처	데몬(dockerd) 기반, 클라이언트-서버 구조	데몬리스(daemonless) , fork-exec 방식	데몬 없음, 단발성 CLI 도구
root 권한	기본적으로 root 데몬 필요 (rootless 모드도 지원되지만 후발)	처음부터 rootless 를 우선 설계	root 불필요
컨테이너 실행	가능	가능	불가능 (실행 기능 없음)
Pod 개념	없음 (Kubernetes와 별개)	Pod 개념 지원 (K8s와 유사한 방식으로 컨테이너 그룹화 가능)	없음
CLI 호환성	표준	Docker CLI와 거의 동일한 명령어(<code>podman run</code> , <code>podman build</code> 등)	별도 전용 명령어
이미지 포맷	OCI/Docker 이미지 지원	OCI/Docker 이미지 지원	OCI/Docker 이미지 지원

2. OCI Image vs Docker Image

- OCI (Open Container Initiative)
Docker가 컨테이너 생태계를 독점하던 시기, 특정 회사에 종속되지 않는 표준을 만들기 위해 2015년 리눅스 재단 산하에 설립된 표준화 단체. 다음 세 가지 주요 스펙을 정의함.

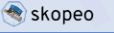
스펙	내용
Image Spec	컨테이너 이미지의 포맷(레이어, 메타데이터 구조) 표준
Runtime Spec	컨테이너를 실제로 실행하는 런타임의 동작 방식 표준 (예: runc)
Distribution Spec	이미지를 레지스트리에 push/pull하는 API 표준

- Docker Image와 OCI Image는 사실상 동일함.
 - Docker는 Docker Image Manifest V2를 OCI에 기증했고, OCI Image Spec은 Docker의 포맷을 기반으로 만들어짐.
 - docker build로 만든 이미지는 내부적으로 OCI 호환 포맷을 따름.

문제 및 풀이

kubernetes.io
Keyword

TASK
A Dockerfile has been prepared at [build/Dockerfile](#)
Using the prepared Dockerfile, build a container image with the **name exam** and **Tag 3.0**.
To build use the tool of your choice. **docker**, **skopeo** or **podman** and **img**.
Using the tool of your choice. export the built container image in **OCI** or **Docker image format** and store it at [export/exam-3.0.tar](#)

	빌드, 실행, 전송, 네트워킹, 볼륨
(Daemon)	항상 실행중 도커 엔진 죽으면, 도커를 통해 실행된 컨테이너가 모두 죽음 여러 컨테이너 동시 빌드시 병목현상 발생
(Root)	루트로 설치되서, docker 실행이 필요한 유저한테 권한 부여
	빌드, 실행, 전송
(Daemonless)	사용할 때만 실행됨 컨테이너는 Podman과 상관없이 독립적으로 실행 여러 컨테이너 동시 빌드시 병목현상 발생안함
(Rootless)	사용자 권한으로 podman을 설치하고 사용
	전송, 포맷 변환

- Docker file은 build/Dockerfile
- image name exam, Tag3.0인 container image 빌드
 - sudo docker build -t exam:3.0 build
 - 마지막 build는 Dockerfile이 위치한 폴더명

- `sudo docker images`
빌드한 이미지 조회
- `podman build -t exam:3.0 build`
podman은 사용자 권한으로 사용되므로 `sudo`가 필요하지 않음.
- `podman save exam:3.0 -o export/exam-3.0.tar --format oci-archive`
format 부분이 없다면 docker format으로 저장됨.
- 빌드한 이미지는 `export/exam-3.0tar`에 저장
 - `sudo docker save exam:3.0 -o export/exam-3.0.tar`
 - Docker Format → OCI로 변환하려면 Skopeo의 포맷 변환 기능 사용 필요
`sudo skopeo copy docker-archive:export/exam-3.0.tar oci-archive:export/exam-3.0-oci.tar`
 - `sudo docker buildx build -t exam:3.0 build -o type=oci, dest=export/exam-3.0.tar`
`docker build`와 OCI 포맷변환을 진행하는 명령어로 앞으로 `docker buildx`만 지원 (?)